



Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación
ISIS-2111 Elementos Esenciales de Lenguajes de Programación

Proyecto 4

Verilang

Integrantes:

Isabela Archbold Cardenas - 202420166

Sebastián Lara Urquijo - 202420591

25 de mayo de 2026

Índice

1. Objetivos del Proyecto	2
2. Arquitectura General del Proyecto	3
3. Desarrollo del Backend en Rascal	4
3.1. RunnerJson.rsc	4
3.2. Manejo de Errores	4
4. Desarrollo de la Interfaz Kotlin	5
4.1. Compose Desktop	5
4.2. LangService.kt	5
4.3. Integración entre Rascal y Kotlin	5
4.4. Problemas Encontrados	5
5. Configuración del Entorno	7
6. Pruebas Realizadas	8
6.1. Archivo ejemplo.vl	8
6.2. Archivo error.vl	9
6.3. Archivo TypeError.vl	9
7. Evidencias del Proyecto	10
7.1. Interfaz Principal	10
7.2. Ejecución Exitosa de ejemplo.vl	11
7.3. Prueba Correcta con Good.vl	11
7.4. Manejo de Error de Parsing	12
7.5. Manejo de Error de Tipos	12
7.6. Ejecución de DomainIndex.vl	12
8. Conclusiones	14

1. Objetivos del Proyecto

El objetivo principal del proyecto fue integrar el lenguaje Verilang desarrollado en Rascal con una interfaz gráfica de escritorio construida en Kotlin.

Los objetivos específicos fueron:

- Implementar un runner en Rascal capaz de procesar archivos .vl.
- Construir una interfaz gráfica utilizando Kotlin.
- Integrar Rascal y Kotlin mediante ejecución automática del shell de Rascal.
- Mostrar resultados de parsing, chequeo de tipos y semántica.
- Manejar errores de manera controlada utilizando JSON.
- Validar distintos archivos de prueba del lenguaje Verilang.

2. Arquitectura General del Proyecto

El proyecto quedó dividido en dos componentes principales:

- Backend del lenguaje desarrollado en Rascal.
- Interfaz gráfica desarrollada en Kotlin.

La arquitectura general del sistema funciona de la siguiente manera:

1. El usuario selecciona un archivo `.vl` desde la interfaz gráfica.
2. Kotlin ejecuta automáticamente el módulo `RunnerJson.rsc` utilizando el shell de Rascal.
3. Rascal procesa el archivo, construye el AST y genera un resultado serializado en formato JSON.
4. Kotlin recibe el JSON y transforma la información en objetos visuales.
5. Finalmente, la interfaz gráfica muestra el resultado del parsing, chequeo de tipos, validaciones semánticas y pretty printing del programa.

3. Desarrollo del Backend en Rascal

El backend del proyecto 4 fue desarrollado completamente en Rascal y utilizó los componentes construidos en proyectos anteriores del curso, como la gramática, el parser y la representación del AST.

Sin embargo, para este proyecto fue necesario extender el lenguaje con nuevos mecanismos de integración que permitieran la comunicación directa con la interfaz gráfica desarrollada en Kotlin.

3.1. RunnerJson.rsc

Durante esta etapa se desarrolló `RunnerJson.rsc`, módulo encargado de conectar el backend implementado en Rascal con la interfaz gráfica construida en Kotlin.

Este archivo automatiza todo el flujo de ejecución del lenguaje. A partir de un archivo `.vl` seleccionado desde la interfaz gráfica, `RunnerJson.rsc` se encarga de leer el programa, ejecutar el parser, construir el AST y generar un resultado serializado en formato JSON.

Adicionalmente, el módulo también genera el pretty printing del programa y organiza toda la información necesaria para ser visualizada posteriormente dentro de la interfaz gráfica.

La utilización de JSON permitió desacoplar completamente la lógica del lenguaje de la interfaz visual, facilitando la comunicación entre Rascal y Kotlin y permitiendo mostrar resultados, estados de ejecución y errores de manera controlada dentro de la aplicación.

3.2. Manejo de Errores

Uno de los mayores retos del proyecto fue el manejo de errores durante la integración entre Rascal y Kotlin.

Inicialmente, ciertos errores internos relacionados con generación del AST o validaciones semánticas producían excepciones que interrumpían la ejecución del sistema. Esto ocasionaba que Kotlin no recibiera JSON válido y, en consecuencia, la interfaz no pudiera mostrar mensajes de error controlados.

Para solucionar este problema, se implementaron excepciones mediante bloques `try catch` dentro de `RunnerJson.rsc`. Gracias a esto, incluso cuando ocurría un error interno en Rascal, el sistema seguía generando respuestas JSON válidas que podían ser interpretadas correctamente desde Kotlin.

4. Desarrollo de la Interfaz Kotlin

4.1. Compose Desktop

La aplicación permite seleccionar archivos .vl, ejecutar programas Verilang y visualizar de forma clara el estado del parsing, chequeo de tipos y validaciones semánticas. Asimismo, la interfaz muestra tanto los resultados exitosos como los mensajes de error producidos por Rascal.

4.2. LangService.kt

El archivo LangService.kt fue el componente encargado de conectar la interfaz Kotlin con el backend implementado en Rascal.

Posteriormente, captura el JSON generado por Rascal y lo transforma en objetos Kotlin que son renderizados directamente dentro de la interfaz gráfica.

4.3. Integración entre Rascal y Kotlin

La comunicación entre Rascal y Kotlin se realizó mediante la ejecución automática del shell de Rascal desde Kotlin utilizando ProcessBuilder.

Kotlin ejecuta dinámicamente RunnerJson.rsc y captura la salida estándar producida por Rascal. Posteriormente, el JSON generado es procesado mediante kotlinx.serialization para convertir la información en objetos Kotlin utilizados directamente por la interfaz gráfica.

4.4. Problemas Encontrados

Uno de los primeros problemas estuvo relacionado con la estructura del proyecto y las rutas utilizadas por Rascal. Inicialmente, algunas carpetas fueron configuradas incorrectamente, lo que impedía que Rascal encontrara correctamente los módulos del lenguaje.

Posteriormente surgieron problemas relacionados con la ejecución automática del shell de Rascal desde Kotlin. En particular, fue necesario descargar y configurar correctamente rascal-shell-stable.jar para permitir que Kotlin pudiera ejecutar RunnerJson.rsc dinámicamente.

Adicionalmente, durante la configuración del entorno surgieron incompatibilidades entre Java 26, Kotlin y Gradle. Para solucionar esto, fue necesario instalar Java 21 y confi-

gurar Gradle.

Finalmente, uno de los retos más importantes fue garantizar que Rascal siempre devolviera JSON válido, incluso cuando ocurrían errores internos. Esto requirió rediseñar parte del manejo de excepciones dentro de `RunnerJson.rsc` para evitar que la interfaz gráfica colapsara durante la ejecución.

5. Configuración del Entorno

El entorno de desarrollo requirió la instalación y configuración de múltiples herramientas para garantizar la integración entre Rascal y Kotlin.

Entre las principales herramientas utilizadas se encuentran:

- Java 21
- Gradle
- Kotlin 1.9.22
- Compose Desktop
- Rascal 0.42.0
- rascal-shell-stable.jar

Adicionalmente, fue necesario configurar correctamente las rutas del proyecto y las dependencias de Gradle para permitir la ejecución automática de RunnerJson.rsc desde Kotlin.

6. Pruebas Realizadas

Con el objetivo de validar el correcto funcionamiento de Verilang, se realizaron múltiples pruebas utilizando distintos archivos .vl diseñados para evaluar parsing, chequeo de tipos, validaciones semánticas y estabilidad general del sistema.

Los archivos utilizados fueron:

- **ejemplo.vl**: archivo válido utilizado para comprobar el funcionamiento completo del lenguaje.
- **Good.vl**: archivo adicional válido utilizado para verificar expresiones y operadores correctos.
- **error.vl**: archivo utilizado para validar estabilidad general del parser y procesamiento de programas.
- **TypeError.vl**: archivo diseñado para producir errores de tipos.
- **DomainIndex.vl**: archivo utilizado para validar expresiones relacionadas con dominios.

6.1. Archivo ejemplo.vl

El archivo ejemplo.vl fue utilizado para validar el funcionamiento completo del lenguaje.

La salida esperada fue:

```
Module: Test1
Variables:
  x : int
  y : int
Operator: mayor : int -> int -> bool
Expression: forall y in dominio . x
Expression: forall x in dominio . (neg x and y)
Expression: exists x in dominio . (x > x)
```

El sistema logró parsear correctamente el archivo y mostrar:

- Parse: OK
- Types: OK
- Semántica: OK

6.2. Archivo error.vl

El archivo `error.vl` fue utilizado para validar el manejo de errores de parsing dentro de VeriLang Runner. Para esta prueba se construyó un programa con una expresión incompleta, de manera que el archivo no pudiera ser reconocido correctamente por la gramática del lenguaje.

Durante la ejecución, la interfaz identificó el fallo en la etapa de parsing y mostró los estados *Parse: FAIL*, *Types: FAIL* y *Semántica: FAIL*. Esto permitió comprobar que la aplicación puede reportar errores sintácticos de forma controlada sin detener la ejecución completa del sistema.

6.3. Archivo TypeError.vl

El archivo `TypeError.vl` fue utilizado para validar el manejo de errores relacionados con tipos inválidos dentro del lenguaje.

Inicialmente, este tipo de programas producía excepciones internas dentro de Rascal que impedían generar JSON válido. Posteriormente, gracias al manejo controlado de excepciones implementado en `RunnerJson.rsc`, fue posible capturar estos errores y mostrarlos correctamente desde la interfaz gráfica sin detener la ejecución del sistema.

La ejecución de este archivo permitió comprobar que la integración entre Rascal y Kotlin manejaba correctamente errores semánticos y de tipos.

7. Evidencias del Proyecto

7.1. Interfaz Principal

La siguiente captura muestra la interfaz principal de Verilang Runner desarrollada en Kotlin.

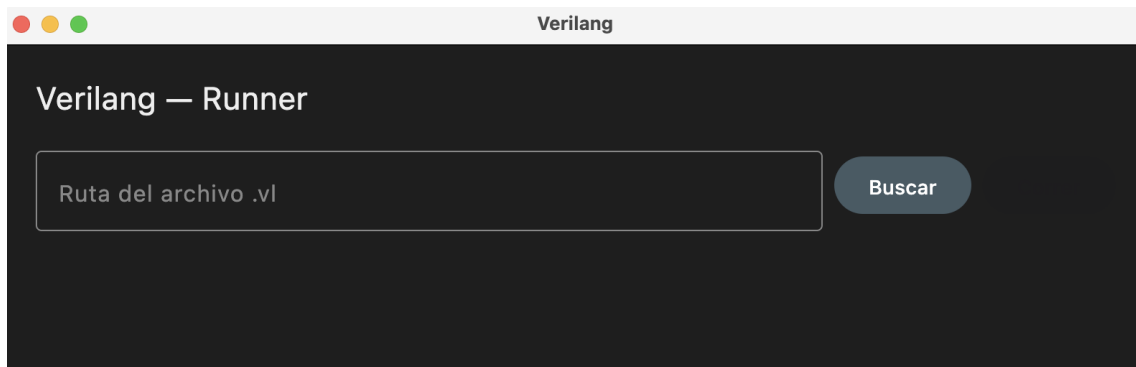


Figura 1: Interfaz principal de Verilang Runner

7.2. Ejecución Exitosa de ejemplo.vl

La siguiente evidencia muestra la ejecución correcta del archivo ejemplo.vl, donde todas las validaciones del lenguaje fueron superadas exitosamente.

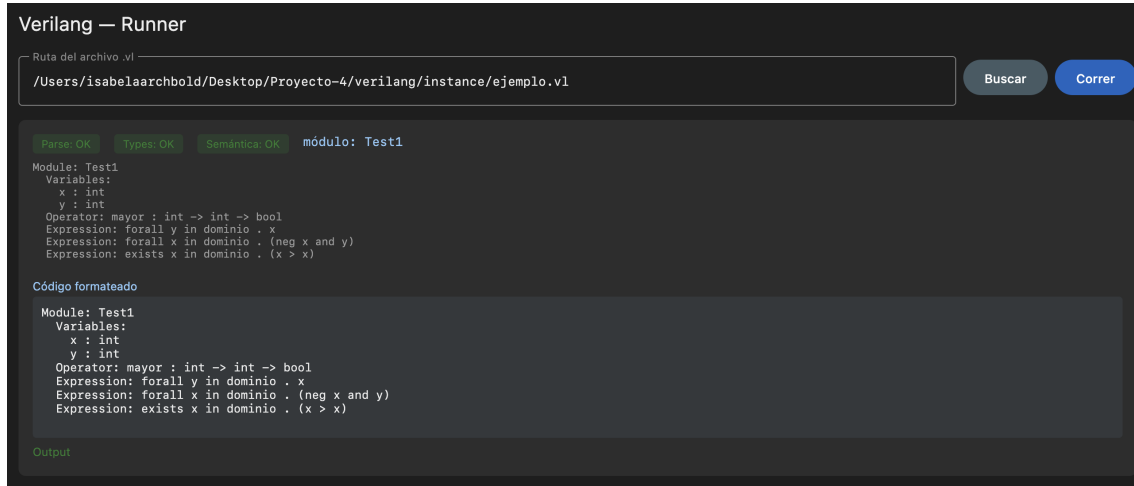


Figura 2: Resultado exitoso del archivo ejemplo.vl

7.3. Prueba Correcta con Good.vl

La siguiente prueba demuestra el funcionamiento correcto del lenguaje utilizando otro archivo válido del proyecto.

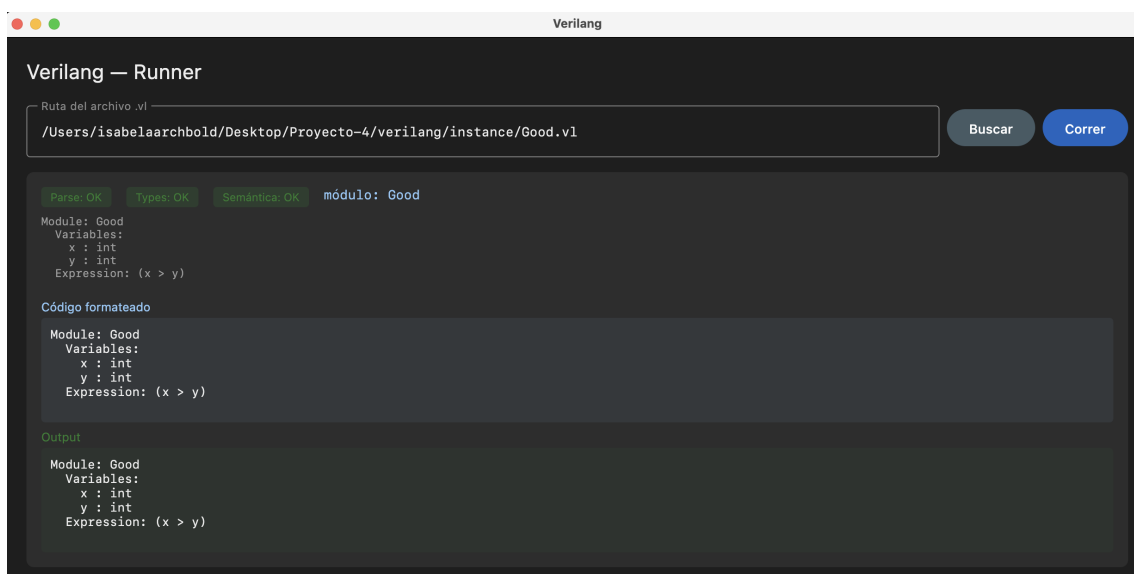


Figura 3: Resultado exitoso del archivo Good.vl

7.4. Manejo de Error de Parsing

La siguiente evidencia muestra cómo la herramienta detecta y reporta errores de parsing sin interrumpir la ejecución de la interfaz gráfica.

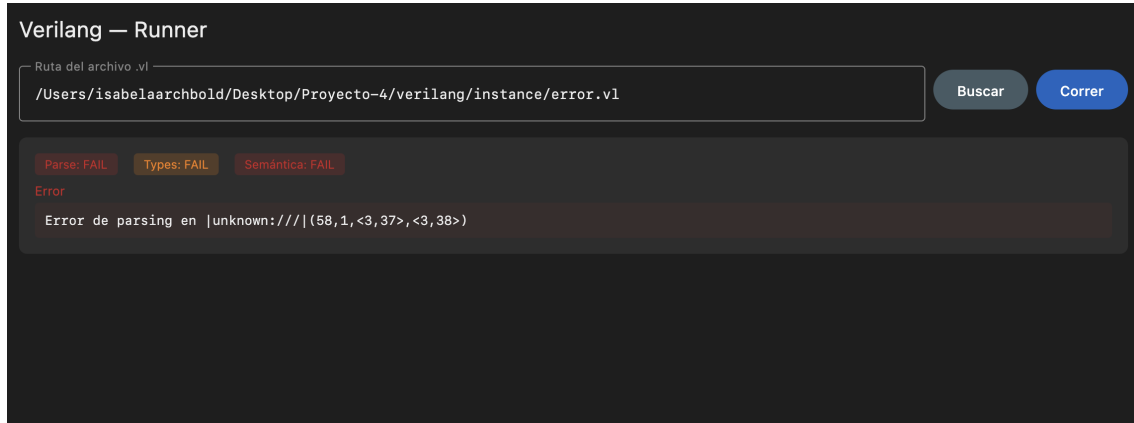


Figura 4: Manejo de errores de parsing utilizando error.vl

7.5. Manejo de Error de Tipos

La siguiente captura evidencia el manejo de errores semánticos y de tipos utilizando el archivo TypeError.vl.

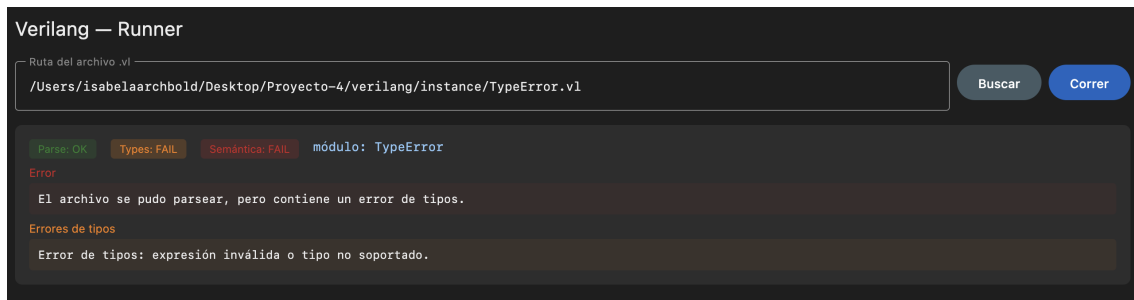


Figura 5: Manejo de errores de tipos utilizando TypeError.vl

7.6. Ejecución de DomainIndex.vl

La siguiente evidencia muestra la ejecución del archivo DomainIndex.vl dentro de la interfaz gráfica y valida el correcto procesamiento de expresiones relacionadas con dominios.

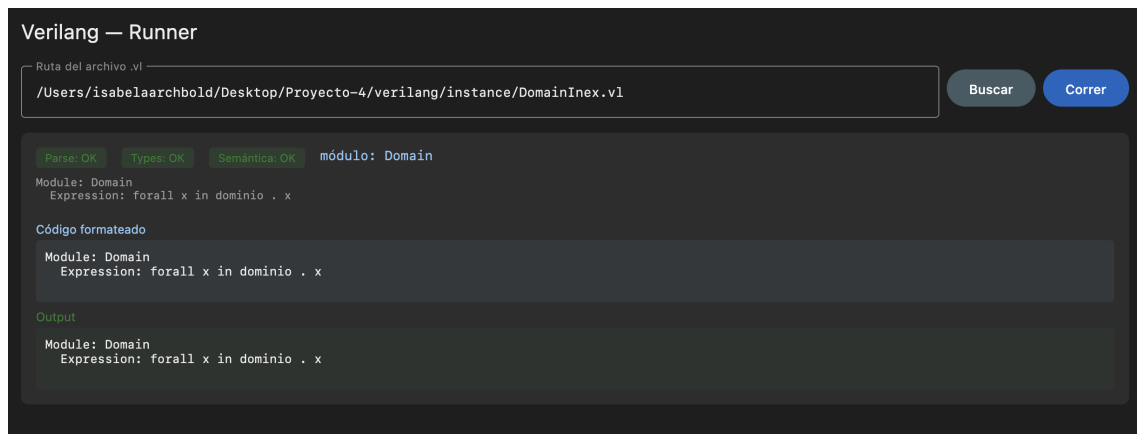


Figura 6: Ejecución del archivo DomainIndex.vl

8. Conclusiones

El Proyecto 4 permitió integrar exitosamente múltiples conceptos vistos durante el curso relacionados con construcción de lenguajes, parsing, representación de ASTs, serialización JSON e integración de herramientas.

Uno de los mayores aprendizajes del proyecto fue la complejidad asociada a integrar tecnologías distintas como Rascal y Kotlin, especialmente en aspectos relacionados con configuración del entorno, rutas del proyecto, manejo de errores y comunicación entre procesos.

Adicionalmente, el proyecto permitió construir una herramienta visual completamente funcional para Verilang, capaz de ejecutar programas reales, mostrar errores controlados y representar resultados de manera amigable para el usuario.

Finalmente, el desarrollo evidenció la importancia de implementar manejo robusto de excepciones y estructuras modulares bien definidas para construir herramientas de lenguajes mantenibles y escalables.